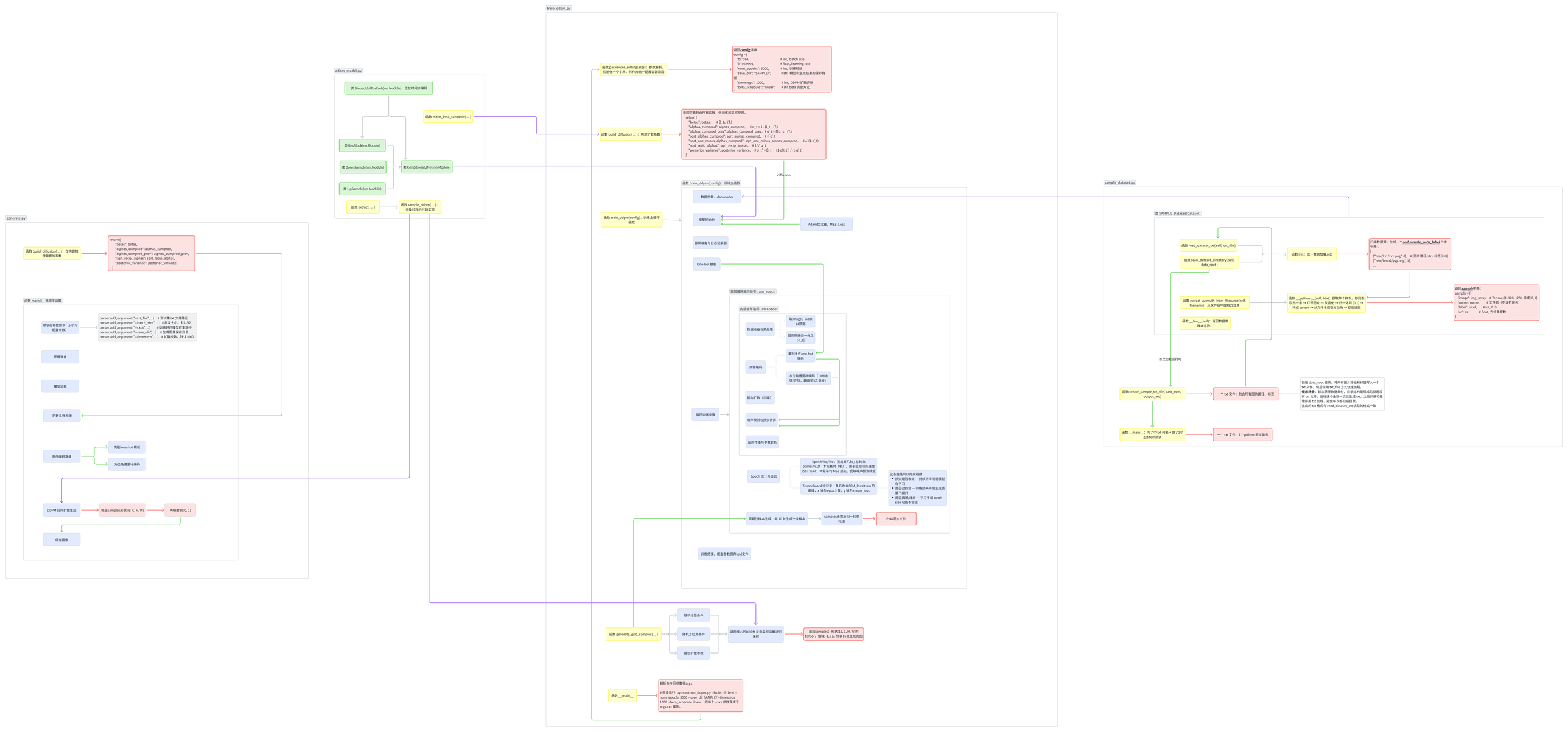
train_ddpm.py
函数 parameter_setting(argv)：参数解析，初始化一个字典。将作为统一配置容器返回
返回config字典：
config = {
"bs": 64, # int，batch size
"lr": 0.0001, # float, learning rate
"num_epochs": 5000, # int，训练轮数
"save_dir": "SAMPLE/", # str，模型和生成结果的保存路径
"timesteps": 1000, # int，DDPM 扩散步数
"beta_schedule": "linear", # str, beta 调度方式
}
函数 build_diffusion(...)：构建扩散系数
返回字典包含所有系数，供训练和采样使用。
return {
"betas": betas, # β_t，(T,)
"alphas_cumprod": alphas_cumprod, # α_t = 1 - β_t；(T,)
"alphas_cumprod_prev": alphas_cumprod_prev, # α_t = ∏α_s，(T,)
"sqrt_alphas_cumprod": sqrt_alphas_cumprod, # √ α_t
"sqrt_one_minus_alphas_cumprod": sqrt_one_minus_alphas_cumprod, # √ (1-α_t)
"sqrt_recip_alphas": sqrt_recip_alphas, # 1/√α_t
"posterior_variance": posterior_variance, # σ_t² = β_t * (1-α(t-1)) / (1-α_t)
}
diffusion
函数 train_ddpm(config)：训练主循环函数
函数 train_ddpm(config)：训练主函数
数据加载，dataloader
模型初始化
Adam优化器，MSE_Loss
目录准备与日志记录器
One-hot 模板
外层循环遍历所有train_epoch
内层循环遍历DataLoader
数据准备与预处理
取image、label
az数据
图像数据归一化之[-1,1]
条件编码
类别条件one-hot编码
方位角傅里叶编码（10维余弦/正弦，最高至5次谐波）
前向扩散（加噪）
噪声预测与损失计算
反向传播与参数更新
循环训练步骤
Epoch 统计与日志
Epoch %d/%d: 当前第几轮 / 总轮数
ptime: %.2f: 本轮耗时（秒），用于监控训练速度
loss: %.6f: 本轮平均 MSE 损失，反映噪声预测精度
TensorBoard 中记录一条名为 DDPM_loss/train 的曲线，x 轴为 epoch 数，y 轴为 mean_loss
这条曲线可以用来观察：
损失是否收敛 — 持续下降说明模型在学习
是否过拟合 — 训练损失降但生成质量不提升
是否震荡/爆炸 — 学习率或 batch size 可能不合适
周期性样本生成，每 10 轮生成一次样本
samples还需反归一化至[0,1]
PNG图片文件
训练结束，模型参数保存.pkl文件
sample_dataset.py
类 SAMPLE_Dataset(Dataset)
函数 read_dataset_txt(self, txt_file)
函数 scan_dataset_directory(self, data_root)
函数 init：统一数据加载入口
扫描数据源，生成一个self.sample_path_label 二维列表：
[["real/2x1/xxx.png",0], # 图片路径(str), 标签(int)
["real/bmp2/yyy.png",1],
...
]
函数 extract_azimuth_from_filename(self, filename)：从文件名中提取方位角
函数 __getitem__(self, idx)：获取单个样本。即列表取出一条 → 打开图片 → 灰度化 → 归一化到 [0,1] → 转成 tensor → 从文件名提取方位角 → 打包返回
返回sample字典：
sample = {
'image': img_array, # Tensor, (1, 128, 128), 值域 [0,1]
'name': name, # 文件名（不含扩展名）
'label': label, # int, 0~9
'az': az # float, 方位角度数
}
函数 __len__(self)：返回数据集样本总数。
首次加载运行时
函数 create_sample_txt_file(data_root, output_txt)
一个 txt 文件：包含所有图片路径、标签
扫描 data_root 目录，将所有图片路径和标签写入一个 txt 文件，供后续用 txt_file 方式快速加载。
使用场景：首次获得数据集时，目录结构是组成的但还没有 txt 文件，运行这个函数一次性生成 txt，之后训练和推理都用 txt 加载，避免每次都扫描目录。
生成的 txt 格式与 read_dataset_txt 读取的格式一致
函数 __main__：写了个 txt 列表 + 做了1个getitem测试
一个 txt 文件，1个getitem测试输出
ddpm_model.py
类 SinusoidalPosEmb(nn.Module)：正弦时间步编码
函数 make_beta_schedule(...)
类 ResBlock(nn.Module)
类 DownSample(nn.Module)
类 UpSample(nn.Module)
类 ConditionalUNet(nn.Module)
函数 extract(...)
函数 sample_ddpm(...)：去噪过程的付码采用
generate.py
函数 build_diffusion(...)：仅构建推理需要的系数
return {
"betas": betas,
"alphas_cumprod": alphas_cumprod,
"alphas_cumprod_prev": alphas_cumprod_prev,
"sqrt_recip_alphas": sqrt_recip_alphas,
"posterior_variance": posterior_variance,
}
函数 main()：推理主函数
命令行参数解析（5 个可配置参数）
parser.add_argument("--txt_file", ...) # 测试集 txt 文件路径
parser.add_argument("--batch_size", ...) # 批次大小，默认12
parser.add_argument("--ckpt", ...) # 训练好的模型权重路径
parser.add_argument("--save_dir", ...) # 生成图像保存目录
parser.add_argument("--timesteps",...) # 扩散步数，默认1000
环境准备
模型加载
扩散系数构建
条件编码准备
类别 one-hot 模板
方位角傅里叶编码
DDPM 反向扩散生成
输出samples形状 (B, C, H, W)
离映射到 [0, 1]
保存图像
函数 generate_grid_samples(...)
随机标签条件
随机方位角条件
提取扩散参数
调用核心的DDPM 反向采样函数进行采样
返回samples：形状(16, 1, H, W)的tensor，值域[-1,1]，代表10张生成的图
函数 __main__
解析命令行参数得args:
假设运行: python train_ddpm.py --bs 64 --lr 1e-4 --num_epochs 5000 --save_dir SAMPLE/ --timesteps 1000 --beta_schedule linear，把每个 --xxx 参数变成了 args.xxx 属性。